

Prompt: Fixes Módulo de Grupos

Fix 1 — Estado del grupo: quitar del form de creación, agregar en la lista

Problema

El selector de estado está en el paso 1 del form de creación, donde no tiene sentido: un grupo recién creado siempre arranca como "Bloqueado". El estado debería poder cambiarse desde la lista de grupos, sin necesidad de entrar al form de edición.

Paso 1A — En `GroupFormDialog` : quitar el select de estado del paso 1 (solo cuando es creación nueva)

Buscar en el paso 1 del form el bloque del select de estado:

```
<div>
  <Label htmlFor="status">Estado</Label>
  <Select
    value={formData.status}
    onChange={ (value) => setFormData({ ...formData, status: value as GroupSt
  >
    <SelectTrigger data-testid="select-group-status">
      <SelectValue />
    </SelectTrigger>
    <SelectContent>
      <SelectItem value="tentative">Tentativo</SelectItem>
      <SelectItem value="blocked">Bloqueado</SelectItem>
      ...
    </SelectContent>
  </Select>
</div>
```

Reemplazar por: mostrar el select **solo cuando es edición**, y en creación nueva forzar el estado a "blocked" :

```
{isEditing ? (
```

```

<div>
  <Label htmlFor="status">Estado</Label>
  <Select
    value={formData.status}
    onChange={(value) => setFormData({ ...formData, status: value as Group
  >
    <SelectTrigger data-testid="select-group-status">
      <SelectValue />
    </SelectTrigger>
    <SelectContent>
      <SelectItem value="tentative">Tentativo</SelectItem>
      <SelectItem value="blocked">Bloqueado</SelectItem>
      <SelectItem value="confirmed">Confirmado</SelectItem>
      <SelectItem value="inhouse">En Casa</SelectItem>
      <SelectItem value="finished">Finalizado</SelectItem>
      <SelectItem value="cancelled">Cancelado</SelectItem>
    </SelectContent>
  </Select>
</div>
) : null}

```

Y en el estado inicial del `formData` **al crear, asegurar que** `status` **sea siempre**

`"blocked" :`

```

const [formData, setFormData] = useState<Partial<InsertGroup>>>({
  name: group?.name || "",
  contactName: group?.contactName || "",
  contactPhone: group?.contactPhone || "",
  contactEmail: group?.contactEmail || "",
  eventDate: group?.eventDate || "",
  checkInDate: group?.checkInDate || today,
  checkOutDate: group?.checkOutDate || tomorrow,
  status: group?.status || "blocked",    // ← cambiar "tentative" por "blocked"
  releaseDate: group?.releaseDate || "",
  notes: group?.notes || "",
});

```

También en el `useEffect` **que resetea al abrir:**

```

status: group?.status || "blocked",    // ← mismo cambio aquí

```

Paso 1B — En `GroupsPage` **: agregar selector de estado inline en la lista**

En la columna “Estado” de la tabla (donde está `<GroupStatusBadge status={group.status} />`), reemplazar el badge estático por un select inline que llama al PATCH directamente:

```
// Agregar mutation de cambio de estado en GroupsPage:
const updateStatusMutation = useMutation({
  mutationFn: ({ id, status }: { id: string; status: string }) =>
    apiRequest("PATCH", `/api/groups/${id}`, { status }),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/groups"] });
    toast({ title: "Estado actualizado" });
  },
  onError: () => {
    toast({ title: "Error al actualizar estado", variant: "destructive" });
  },
});
```

En la celda de estado de cada fila de la tabla:

```
<TableCell>
  <Select
    value={group.status}
    onChange={(value) =>
      updateStatusMutation.mutate({ id: group.id, status: value })
    }
  >
    <SelectTrigger
      className="h-7 w-36 text-xs border-0 bg-transparent p-0 focus:ring-0"
      data-testid={`select-status-${group.id}`}
    >
      <SelectValue>
        <GroupStatusBadge status={group.status} />
      </SelectValue>
    </SelectTrigger>
    <SelectContent>
      <SelectItem value="tentative">
        <GroupStatusBadge status="tentative" />
      </SelectItem>
      <SelectItem value="blocked">
        <GroupStatusBadge status="blocked" />
      </SelectItem>
      <SelectItem value="confirmed">
        <GroupStatusBadge status="confirmed" />
      </SelectItem>
      <SelectItem value="inhouse">
```

```

      <GroupStatusBadge status="inhouse" />
    </SelectedItem>
    <SelectedItem value="finished">
      <GroupStatusBadge status="finished" />
    </SelectedItem>
    <SelectedItem value="cancelled">
      <GroupStatusBadge status="cancelled" />
    </SelectedItem>
  </SelectContent>
</Select>
</TableCell>

```

Fix 2 — Asignar habitación se refleja automáticamente en Planning y Reservas

Problema

`assignRoomToGroup` ya crea correctamente una reserva real en la DB con `createReservation` + `createGroupReservationLink`. El problema está en el frontend: el `onSuccess` del `assignMutation` solo invalida:

```

queryClient.invalidateQueries({ queryKey: ["/api/groups", group.id] });
queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });

```

Por eso el planning y la lista de reservas no se actualizan hasta que el usuario refresca manualmente.

Fix en el componente de asignación de habitación (dentro de `GroupDetailPage`)

Buscar el `assignMutation` que tiene el `onSuccess` con los dos `invalidateQueries` y agregar las dos líneas faltantes:

```

const assignMutation = useMutation({
  mutationFn: () =>
    apiRequest("POST", `/api/groups/${group.id}/assign-room`, {
      roomId: selectedRoomId,
      guestFirstName,
      guestLastName,
      checkInDate: useCustomDates ? checkInDate : undefined,
      checkOutDate: useCustomDates ? checkOutDate : undefined,

```

```

        agreedRate: useCustomRate ? agreedRate : undefined,
        ratePlanId: block.ratePlanId,
    }},
    onSuccess: () => {
        queryClient.invalidateQueries({ queryKey: ["/api/groups", group.id] });
        queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
        queryClient.invalidateQueries({ queryKey: ["/api/reservations"] }); // ← a
        queryClient.invalidateQueries({ queryKey: ["/api/planning"] }); // ←
        queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] }); // ← a
        toast({ title: "Habitación asignada exitosamente" });
        onSuccess();
        onOpenChange(false);
        setSelectedRoomId("");
        setGuestFirstName("Sin Asignar");
        setGuestLastName("");
        setUseCustomDates(false);
        setUseCustomRate(false);
    },
    onError: () => {
        toast({ title: "Error al asignar habitación", variant: "destructive" });
    },
});

```

Fix adicional — mostrar habitaciones ocupadas por otros grupos también

Actualmente `availableRooms` filtra solo `r.status === "available"`, lo que excluye habitaciones reservadas por otros grupos pero físicamente disponibles. Cambiar el filtro para también excluir por conflicto de fechas en lugar de solo por status:

```

// ANTES — demasiado restrictivo
const availableRooms = rooms?.filter(
    (r) => r.roomTypeId === block.roomTypeId && r.status === "available"
);

// DESPUÉS — mostrar todas del tipo correcto excepto las que tienen reserva activ
const availableRooms = rooms?.filter((r) => {
    if (r.roomTypeId !== block.roomTypeId) return false;
    if (r.status === "maintenance") return false;
    return true; // La validación de conflicto de fechas la hace el backend
});

```

Resumen de cambios

Archivo	Cambio
GroupFormDialog	Quitar select estado en creación, forzar "blocked" por defecto
GroupFormDialog	Mantener select estado solo en edición
GroupsPage	Agregar <code>updateStatusMutation</code> + select inline en columna Estado
GroupDetailPage — <code>assignMutation.onSuccess</code>	Agregar <code>invalidateQueries</code> para <code>/api/reservations</code> , <code>/api/planning</code> , <code>/api/dashboard/stats</code>
GroupDetailPage — <code>availableRooms</code> filter	Mostrar habitaciones del tipo correcto, no solo <code>status === "available"</code>